NEI Course Design Template

NEI Course Design Template

Course Basics

Course Title: My First Web App

Course Length (6 weeks or 12 weeks): 6 weeks

Accreditation and external credibility (if applicable): Certificate options through codecademy

Existing MIT courses that NEI can build on (if applicable):

Course(s): web.lab, 6.4500

How the course(s) can be used: web.lab is a 4 week IAP course, with teaching primarily within the first two weeks. It has online powerpoints and synchronous projects. 6.4500 is Design for the Web: Languages and Interfaces. It is a semester-long course with two weekly lectures followed by a studio/implementation project. Available lecture videos, etc.

Tech Stack: HTML, CSS, Javascript, Django

Course Planning (Goals, Methods, & Metrics):

Course learning goals: incorporate NEI institutional learning goals & any additional objectives.

Learning goals	Course-specific description
Employable	Students will gain hands-on experience with the core tools of web development (HTML, CSS, JavaScript, HTTP, WebSockets, and authentication) and will build a project they can present and explain to others.
Discerning	Students will learn to distinguish good design from bad design by recognizing code that is clean, safe, easy to maintain, and reduces duplication
Savvy	Students will learn how the web operates under the hood through client-server communication, real-time protocols, and authentication.
Life-long Learning	Students will build the confidence to learn new web development tools, debug unfamiliar problems, and extend their projects independently after the course ends.
Citizens	Students will learn in an environment that helps them cultivate responsibility and empathy. They will use basic UX/UI principles to think critically about how they design user accounts, handle user data, and apply basic privacy and security measures to protect the people who use their applications.

Course-design elements & methods: incorporate NEI institutional teaching methods & additional course elements.

Elements & methods	Course-specific description
Guided Discovery	Students will encounter new web development concepts through a visible problem, failure, or artifact they cannot fully explain yet. In class, students will inspect and experiment with starter code, identify issues, make predictions about how the code is structured, and try to fix or modify it before receiving a short lecture that explains the underlying concept and connects it to broader web development principles.
Project-based learning	Students will build one to two cumulative web apps across the course. The project will begin as a simple webpage and gradually add styling, interactivity, client-server communication, real-time updates, user accounts, and basic privacy and security practices.
Replicating workplace workflows	Students will use tools and practices that mirror real web development environments, including GitHub, browser developer tools, code reviews, README documentation, peer feedback, and project demos.

Metrics: possible ways to assess students' level of mastery for course concepts.

Metric	Description
Weekly Quizzes	Students complete a short quiz covering the week's central concepts and code-reading skills. Students who score below the target complete corrections or retry selected questions.
Peer Feedback	Students participate in at least two structured review sessions. Each student identifies one strength, one specific issue, and one actionable recommendation in a classmate's project. Students document at least one revision made in response to feedback.
Weekly Project Milestones	Students submit one cumulative project milestone each week. Each milestone is evaluated for completion, functionality, connection to the week's learning goals, and evidence that feedback from previous weeks was applied.
Code Checks	Instructor review and automated checks evaluate whether the code runs correctly, follows the required structure, avoids unnecessary duplication, and meets basic accessibility and security expectations. Students must correct critical errors before moving to the next milestone.
Final Project Rubric	The final application is assessed across five areas: functionality, frontend design and responsiveness, backend and data handling, code quality, and security/accessibility.

Basic Outline

Topic A

Descriptive Title: Introduction to Static Web Development and Styling
Length: 2 weeks
Motivating question or goal: How can we use HTML and CSS to create a structured, accessible, and responsive website?
Final artifact: A responsive, multi-page website that demonstrates semantic HTML, consistent CSS styling, accessibility, and responsive design techniques.

Topic A.1: Basic HTML and Styling:

- Explain the purpose of HTML and identify the basic structure of an HTML document.
- Use tags and elements to organize webpage content.
- Properly nest HTML elements.
- Distinguish among attributes, classes, and IDs.
- Use and semantic HTML elements to organize content.
- Add links between pages and external websites.
- Apply basic inline styling.
- Explain why excessive inline styling can make code repetitive and difficult to maintain.
- Apply introductory accessibility practices, including alternative text, semantic elements, descriptive links, and clear heading structures.

Topic A.2: Introduction to CSS:

- Apply basic styling properties, including fonts, colors, spacing, borders, and backgrounds.
- Style text, images, navigation bars, and footers.
- Explain and use the CSS box model.
- Select HTML elements using Element, class, id, and attribute selectors
- Explain how the CSS cascade, specificity, inheritance, and source order determine which styles are applied.
- Use CSS selectors to style multiple elements efficiently.
- Refactor inline styles into reusable CSS rules.
- Distinguish between shorthand and longhand CSS properties.
- Use common positioning methods (Static, Relative, Absolute and Fixed)
- Explain the purpose of the display property.
- Apply basic image-positioning techniques

Topic A.3: Advanced HTML & CSS Creating Responsive Websites:

- Identify basic principles of effective visual and interface design.
- Evaluate websites using design case studies.

- Create a basic webpage mockup (can be by hand, using Figma or a similar design tool).
- Recognize the limitations of manually positioning every webpage element.
- Explain how responsive design allows a website to adapt to different devices and screen sizes.
- Use Flexbox to create one-dimensional layouts.
- Use CSS Grid to create two-dimensional layouts.
- Use Bootstrap components and its responsive grid system.
- Combine Flexbox, Grid, and Bootstrap appropriately within one website.
- Apply property inheritance to reduce repetitive styling.
- Translate a basic design mockup into a functioning webpage.

Topic B

Descriptive Title: Creating Dynamic Websites
Length: 3 weeks
Motivating question or goal: How can we make websites interactive, store information, and respond to users?
Final artifact: A functional full-stack web application that includes interactive browser behavior, server communication, persistent data, user input, and basic account functionality.

Topic B.1 Introduction to Interactivity with JavaScript: Two Weeks

- Explain the role of JavaScript in web development.
- Use JavaScript variables, data types, operators, functions, conditionals, and loops.
- Explain block and lexical scope.
- Recognize hoisted declarations and the risks of assigning values to undeclared variables.
- Write information to the browser console for testing and debugging.
- Predict the results of short JavaScript programs.
- Create and manipulate strings, arrays, objects, and JSON-compatible data.
- Distinguish between DOM elements and other types of nodes.
- Select HTML elements using JavaScript.
- Modify webpage content, attributes, and styles through JavaScript.
- Add and respond to browser events.
- Explain the Document Object Model as a tree.
- Use an interactive HTML tree generator to visualize the DOM.
- Add, remove, and modify DOM elements.
- Explain what happens to removed DOM elements.
- Recognize that browser-only changes are generally lost when a page reloads.
- Identify the need for servers and databases when information must persist.

Topic B.2: Servers One Week

- Distinguish between client-side and server-side code.
- Describe the request–response cycle.
- Explain the roles of HTTP and HTTPS.
- Compare standard HTTP communication with persistent WebSocket connections.
- Distinguish among common request methods, including GET, POST, PUT, and DELETE.
- Send and receive structured data using JSON.
- Use fetch to communicate with a server.
- Explain asynchronous execution.
- Use promises and async/await to manage asynchronous operations.
- Describe the role of a web framework.
- Create a basic Django project and application.
- Define routes, views, and endpoints.

- Connect frontend actions to backend functionality.
- Explain the purpose of a database.
- Store, retrieve, update, and delete basic application data.
- **Use in memory state instead of using a database**
    - **Could have a separate topic for databases**
- **Possibly drop the database in favor of websockets**

Topic B.3: Accounts, User Input, and Real-Time Behavior One Week

- Create and process HTML forms.
- Validate and sanitize user input.
- Explain the difference between authentication and authorization.
- Create basic account registration, login, and logout functionality.
- Restrict application features based on a user's identity or permissions.
- Explain why passwords should never be stored as plain text.
- Identify common web security concerns.
- Explain when real-time communication is useful.
- Describe how WebSockets support persistent, two-way communication.
- Implement or explore a basic real-time feature, such as live updates, notifications, or messaging.

Drop topic C, likely won't fit with time constraints, add to B, B.1 could take two weeks

Topic C

Descriptive Title: **Additional Tools for Web Development (Emphasis critical assessment)**
Length: 1 week
Motivating question or goal: How can modern tools support web development while still requiring developers to evaluate their output critically?
Final artifact:

Topic C.1: Additional Tools for Web Development

- Use Figma or a similar design platform to create and revise webpage designs.
- Explore tools that convert visual designs into frontend code.
- Use an AI assistant, such as Claude, to brainstorm, generate, explain, debug, or refactor code.
- Review AI-generated code for correctness, security, accessibility, and maintainability.
- Identify situations in which generated code should not be accepted without revision.
- Compare different programming languages, frameworks, libraries, and technology stacks used in web development.
- Explain how the tools taught in the course fit into the broader web development ecosystem.
- Recognize that different projects may require different frontend, backend, database, and deployment technologies.
- Select tools based on project requirements rather than novelty or convenience.
- Reflect on the importance of understanding foundational concepts even when using automated development tools.

Week-by-Week Breakdown

Week	Topic	Focus	Activities	Assignments or Products
1	A.1 & A.2	HTML foundations and introductory CSS	Inspect a poorly structured webpage; predict how nested HTML will appear; practice tags, attributes, classes, IDs, links, images, and semantic elements; use browser developer tools; refactor inline styling into an external stylesheet.	**Static Website Milestone 1:** Create the HTML structure for a multi-page personal website, including navigation, images, semantic elements, and basic accessibility features.
2	A.2 &A.3	Responsive styling and visual design	Evaluate examples of effective and ineffective website design; create a basic Figma mockup; practice the box model, selectors, positioning, Flexbox, Grid, and Bootstrap; test layouts at different screen sizes.	**Static Website Milestone 2:** Style and revise the website so that it is visually consistent, accessible, and responsive on desktop and mobile screens.
3	B.1	JavaScript and browser interactivity	Predict the output of short JavaScript programs; use the console for debugging; manipulate strings, arrays, and objects; visualize the DOM as a tree; select and modify elements; create event listeners and interactive page features.	**Interactive Website Milestone:** Add at least two interactive features, such as a quiz, filter, counter, form response, theme switcher, or dynamically updated content. Explain what happens to the changes when the page reloads.
4	B.2	Servers, APIs, and persistent data	Model the request-response cycle; inspect browser network requests; compare clients and servers; practice GET, POST, PUT, and DELETE requests; use	**Full-Stack Milestone 1:** Connect the frontend to a Django server and database. Users must be able to

			JSON, fetch, promises, and async/await; create Django routes, views, models, and endpoints.	create and retrieve information that remains available after the page reloads.
5	B.3	User accounts, security, and real-time behavior	Build and process forms; validate user input; distinguish authentication from authorization; examine password storage and common security mistakes; create protected pages; explore WebSockets through live updates, notifications, or messaging.	**Full-Stack Milestone 2:** Add registration, login, logout, input validation, and at least one user-specific feature. Implement or demonstrate one real-time feature when feasible.
6	C.1	Modern tools, integration, and critical assessment		**Final Artifact:** Submit and present the completed web application, including a README, project demonstration, code repository, and short evaluation of one modern development tool.

Notes (Questions, Concerns, Ideas, Etc.):

"How can we motivate each topic before teaching it? How can we teach from specific-to-general?" by July 9

- Meet with prof daniel jackson
- Possible implementations:
    - Given base code -> ask student to edit to make more applicable to themselves (editing tasks)
        - Personal web page
        - Making a recipe
    - Giving messy code -> asking students to identify whats wrong/repetitive
        - Can introduce css classes
    - Identify the error tasks -> something isn't working? Fix this
        - For later in the course
    - Different code examples
- Homework:
    - Implement this into own project/webpage
- Design of software/good structure and database
- Django or Flask server vs client
- Exploring general teaching methods for the class
- Lectures bridging to the next lab
- Code checks for labs?
- Guided discovery in psets as well?
- Prettier for code formatting
- Autograding for projects
- branching
- gradescope for autograding
- Naming labs -> classes
- Use of the word units/modules
- https://csszengarden.com/
- 00A -> video
- Activity, test, homework

- Writing notes for the next person ->
    - More research for different course designs beyond PRIMM/parson's problems

Guided Discovery Labs

Topic A: Static Web Development and Styling

Week 1: HTML and CSS Foundations

Lab 1: What Is HTML?

Students explore HTML syntax, tags, elements, attributes, and nesting by editing and repairing a basic webpage.

Lab 2: Organizing and Connecting Webpages

Students use semantic HTML, `<div>` elements, headings, links, and navigation to create an accessible multi-page website.

Lab 3: Why Do We Need CSS?

Students experiment with inline, internal, and external styling and discover why separating HTML and CSS makes websites easier to maintain.

Lab 4: CSS Selectors and the Cascade

Students use element, class, ID, and attribute selectors and investigate how source order, specificity, and inheritance determine which styles apply.

Lab 5: The Box Model and Visual Styling

Students style fonts, colors, images, spacing, borders, and footers while using the box model and refactoring repetitive CSS.

Week 2: Layout, Responsiveness, and Design

Lab 6: Designing Before Coding

Students analyze effective and ineffective websites, learn basic design principles, and create a simple webpage mockup in Figma.

Lab 7: Positioning and Page Flow

Students explore block and inline display, static, relative, absolute, and fixed positioning, and limited uses of floats.

Lab 8: Building Layouts with Flexbox

Students use Flexbox to create navigation bars, aligned content, and responsive one-dimensional layouts.

Lab 9: Building Layouts with CSS Grid

Students use Grid to organize content across rows and columns and compare when Grid or Flexbox is the better layout tool.

Lab 10: Bootstrap and Responsive Website Build

Students explore Bootstrap, customize framework components, and complete a responsive multi-page website using the tools learned throughout the unit.

Deliverable: Personal Portfolio Website

- Requirements: Homepage, About page, projects/interests/experience page, navigation between pages, descriptive links and image alternative text, appropriate use of classes and IDs, correct nesting and file organization, visually appeasing design

Topic B: Creating Dynamic, Interactive Websites

Week 3: JavaScript and Browser Interactivity

Lab 11: Introduction to JavaScript

Students compare JavaScript with Python and practice variables, data types, conditionals, functions, and console output.

Lab 12: JavaScript Objects and Data

Students work with strings, arrays, objects, and JSON while identifying important differences between JavaScript and Python.

Lab 13: Understanding the DOM

Students inspect the DOM as a tree and use JavaScript to select and modify webpage elements.

Lab 14: Events and User Input

Students use event listeners, buttons, inputs, and forms to make a webpage respond to user actions.

Lab 15: Building a Browser-Only Application

Students combine JavaScript and DOM manipulation to build a small interactive application, then observe that its data disappears when the page reloads.

Deliverable: Interactive frontend prototype that works until the page is refreshed

Week 4: Servers and Persistent Data

Lab 16: Client versus Server

Students investigate which code runs in the browser, which code runs on a server, and how the two communicate through requests and responses.

Lab 17: HTTP and Web Requests

Students explore URLs, HTTP and HTTPS, and GET, POST, and PUT requests by examining how data moves between a client and server.

Lab 18: Fetch, JSON, and Asynchrony

Students use `fetch` to request data and learn how promises and asynchronous code allow the browser to continue working while waiting for a response.

Lab 19: Introduction to Django

Students create a basic Django project and use routes and views to return webpages or data from a Python server.

Lab 20: Forms and Databases

Students submit form data to Django and store information in a database so that it remains available after the page reloads.

Deliverable: A Django application that saves and retrieves data

Week 5: Accounts, Security, and Real-Time Behavior

Lab 21: Creating and Managing Data

Students build basic create, read, update, and delete features that allow users to manage stored information.

Lab 22: User Accounts and Authentication

Students use Django's built-in tools to create registration, login, and logout functionality.

Lab 23: Authorization and User-Specific Content

Students control which pages and data a user can access and distinguish authentication from authorization.

Lab 24: Web Security

Students investigate common security risks involving passwords, forms, permissions, user input, and generated code, then improve an insecure application.

Lab 25: Real-Time Communication with WebSockets

Students compare WebSockets with traditional HTTP requests and build or examine a simple real-time feature such as chat, notifications, or live updates.

Deliverable: Functional full-stack web application with persistent data and user accounts

Topic C: Additional Tools for Web Development

- **More Designing with Figma**
- **Using AI as a Development Assistant**
- **Avoiding Cookie-Cutter AI Design**
- **Evaluating and Refactoring Generated Code**
- **Exploring Other Web Development Stacks**

Teaching Strategies

Programming Teaching Methods and Research Notes

PRIMM: Predict, Run, Investigate, Modify, Make

PRIMM emphasizes teachers and students talking about programming and encourages students to discuss code before writing it.

Predict

- Students read a high-quality sample program created by the teacher.
- Students learn how to read code, which is an essential first step before writing code.

Run

- Students run the actual code.

Investigate

- Students answer carefully constructed questions that draw out important learning points.

Modify and Make

- Students modify existing code and eventually create something themselves.
- This helps students take ownership of what they make and gain experience writing code.

The study found a significant increase in outcomes for the experimental group.

Pair Programming:

- Two people simultaneously work on a single software project.
- The driver controls the mouse and keyboard.

- The navigator reviews the written code and keeps track of what has been completed compared with the design.
- Students swap roles regularly.
- Pair programming has been associated with increased learning and improved code quality among university students, but it requires careful implementation to ensure success.
- The greatest increases occurred when a confident partner was paired with a partner who had less programming experience.
- On the contrary, some research showed less work completed overall and no increase in overall progression during a summer school implementation.

Good to try it out

How many methods to try to hold together at one time?

Maybe implement in a few labs -> could help with communicating and talking about code amongst each other

Possible Implementation Differences

- In the summer school implementation, students received new partners every day.
- In the other implementations, students had the same partner for the entire project.
- In the summer school implementation, roles were swapped every five minutes.
- In the school-year implementation, roles were swapped every 20 minutes.
- Switching too often may make it harder for students to progress through the project.

Peer Instruction

- There is evidence that a specific implementation of peer instruction increases learning.
- Learners are provided with questions and submit their answers using flashcards or an online voting system.

- They then discuss their thinking with peers before resubmitting their answers.
- Teachers review the first and second answers and provide further support if needed.
- This may be more like “peer-led” instruction, where classroom results are aggregated and teachers target the areas that need additional focus.
- There is little evidence of improvement in final exam scores.

Think-Pair-Share

- Increased learning has been found in undergraduate programs.
- It had little effect on middle school students.
- It helps increase participation.

Live Coding

- Students observe the teacher completing an activity while talking through their thought process.
- The teacher must carefully select examples that introduce new concepts.
- Live coding should reveal the demonstrator’s thinking through “thinking out loud.”
- Returns increased when students coded along with the instructor.

Completed by TAs after each activity, paired with the clicker questions/peer instruction to further understanding among students

Theoretical Background: Cognitive Apprenticeship

Modeling

- The instructor leads a live-coding session and demonstrates a task from scratch.

Scaffolding

- The teacher gives students a task and provides the needed scaffolds or code skeleton.

Fading

- The teacher gradually removes the scaffolds so students learn to complete the task independently.

Areas Examined in the Implementation

1. Positives: Benefits of live coding
2. Debugging: Live coding as a way to learn debugging
3. Negatives: Areas that could be improved during live coding
4. Long-term benefits: Learning useful skills through live coding

Pattern-Oriented Instruction

- Reduces cognitive load by allowing students to think about a pattern as a "black box" that meets a particular goal.
- Helps students break problems into parts and build potential solutions.
- Students are introduced to examples of a pattern, with examples becoming more complex over time.
- Students examine similarities and differences to discover how patterns are used or misused.
- Students also think about alternative patterns.

Targeted Tasks

Targeted tasks may include:

- Debugging
- Sabotage
- Annotation
- Fill-in-the-gap activities
- Parsons problems

Other activities include:

- Predicting what code will do
- Matching designs to programs
- Investigating and fixing buggy code
- Sabotaging code for peers to fix

- Annotating code to explain what it is intended to do

Parsons Problems-

Parsons problems are programming puzzles in which learners arrange scrambled lines or blocks of code into the correct logical order.

Possible variations include:

- Distractors: Superfluous lines of code containing semantic errors
- Faded problems: Students increasingly complete portions of the code themselves
- Adaptive problems: The difficulty is dynamically controlled based on student performance

Parsons problems may improve engagement and pattern recognition.

There is a lack of replicated research on their effectiveness.

Sources

PRIMM:
https://suesentance.net/wp-content/uploads/2020/02/teaching_computer_programming_with_primm__a_sociocultural_perspective_author_copy.pdf

Teaching Programming in Schools: Pedagogy Review:
https://www.raspberrypi.org/app/uploads/2021/11/Teaching-programming-in-schools-pedagogy-review-Raspberry-Pi-Foundation.pdf

Parsons Problems:
https://acelab.berkeley.edu/wp-content/papercite-data/pdf/parsons-chi2021.pdf

Live Coding:
https://pages.cs.wisc.edu/~gerald/papers/LiveCoding.pdf

Teaching Methods and Class Structure

Instructional Frameworks

Teaching Method	How It Would Be Used
PRIMM	Students move through Predict, Run, Investigate, Modify, and Make. They first read and discuss code, then run it, investigate how it works, modify it, and eventually create something of their own. Scaffolding is built into the progression, with support gradually removed as students move toward making.
Live Coding and Cognitive Apprenticeship	The instructor completes a task while thinking out loud and showing how they make decisions, identify mistakes, and debug. Students may code along, complete a similar task with support, and then complete one more independently.
Pattern-Oriented Instruction	Students examine multiple examples, identify similarities and differences, and learn reusable HTML, CSS, or programming patterns. Examples become more complex over time.
Peer Instruction	Students answer a question individually, discuss their reasoning with a partner or group, and then answer again. The instructor uses the class responses to decide which concepts need more explanation.
Pair Programming	Two students work on one project. The driver controls the keyboard and mouse, while the navigator reviews the code and tracks progress. Students switch roles during the activity.

Activity Types

These activities could be used within PRIMM, live coding, pattern-oriented instruction, or collaborative work.

Activity Category	Examples
Prediction and Investigation	Predict what code will do, run it, compare the outcome with the prediction, answer questions about how the code works, or match code to a rendered webpage.
Targeted Programming Tasks	Debug incorrect code, repair missing tags or broken selectors, fill in missing code, annotate what code is intended to do, or intentionally sabotage code for a peer to fix.
Parsons Problems	Arrange scrambled lines or blocks of code into the correct order. Problems may include incorrect lines as distractors or gradually require students to write more of the code themselves.
Comparison Activities	Compare inline, internal, and external CSS; compare different solutions to the same problem; or examine multiple examples to identify a common pattern.
Modification and Refactoring	Change existing code, replace repetitive styles with reusable CSS rules, or improve the structure and readability of a program.
Mini Build Challenges	Create a small webpage or feature using required concepts after students have studied and modified examples.
Reflection and Explanation	Explain what was changed, why a solution works, what was learned, or what remains confusing.

Collaborative Structures

Structure	Example
Think-Pair-Share	Students first think about a coding question independently, discuss it with a partner, and then share their reasoning with the class. Sharing can be via clickers/more streamlined ways to get everyone's input.

Collaborative Debugging	Students work together to identify, explain, and repair a problem in the code. Focus on communication/explaining errors to another person.
Peer Review	Students review a partner's code/styling and provide feedback.
Pair Programming	Students work in driver and navigator roles on the same coding task or project.

Class Norms and Implementation Decisions

These are not necessarily teaching methods, but they affect how the methods work in practice.

- Students should discuss and read code before being expected to write it independently.
- Students should explain their reasoning rather than only provide the correct answer.
- Mistakes and debugging should be treated as a normal part of programming.
- Students should gradually receive less starter code and instructional support.
- Collaborative work should include clear roles and expectations.
- Students should regularly reflect on what they changed and why.
- Students should compare multiple possible solutions instead of assuming there is only one correct approach.

Testing

- Get people to talk through the thinking process and where they got stuck etc.
- Record as well -> easier to take notes and analyze as well
-

Trial Runs

Lab	Approx. time	Problems observed	Revisions made	Further changes
Lab 1	40 mins	- Repetition in instructions - Wording to make instructions more clear (changing a set of h1s vs one h1)	- Fixed wording and deleted extra repetition	
Lab 2	40 minutes	- Difficulty understanding what styling to use in the freestyle portion - More explicit explanations -> should they be in the labs themselves or directed for the teachers to help make the connections	- Adding a basic styling resource for students to refer to	- Missing an introduction to flie storage/linking but may not be needed if students have prior python knowledge - Providing basic styling documentation as homework/resource
Lab 3	30 minutes	- Lab a bit short, needs more content about hierarchy between inline, internal, and external, acts as a natural bridge to the next lab	- Added info about hierarchy across internal, extern	

			al, and inline	
Lab 4	20 minutes	- Needs more content about multiple selectors/combining selectors along with practice	- Added & selectors and or selectors + practice	
Lab 5	40 minutes	- A bit more guidance/rules for the independent designing section - Add in a peer review for this section before allowing	- Added this guidance and a peer review instructions	

Next Steps:

- Move the design introduction lab to the beginning as a primer on analyzing websites, planning layouts, and thinking about users before coding.
- Add an “Apply This to Your Portfolio” section to every lab.
- Make the transition into portfolio work explicit rather than expecting students to transfer the skills independently.
- These labs were tested in a 1:1 format, preform testing more like the actual classroom setup with two students working together